



1. Problem Title & Link

Title: LeetCode 1768 : Merge Strings Alternately

Link: [LeetCode 1768 - Merge Strings Alternately](#)

2. Problem Statement (Short Summary)

You are given two strings:

word1

word2

Merge them by adding letters in alternating order:

- Take first character from word1
- Take first character from word2
- Continue alternately

If one string becomes empty:

- Append remaining characters of the other string

In simple terms:

Mix characters one by one from both strings.

3. Examples (Input → Output)

Example 1

Input:

word1="abc"

word2="pqr"

Output:

"apbqcr"

Example 2

Input:

word1="ab"

word2="pqrs"

Output:

"apbqrs"

Explanation:

Remaining:

rs

gets appended.

**Example 3**

Input:

word1="abcd"

word2="pq"

Output:

"apbqcd"

4. Constraints

$1 \leq \text{word1.length}, \text{word2.length} \leq 100$

word1 and word2 contain lowercase English letters

Special restrictions:

- Must preserve alternating order
- Append remaining characters correctly

5. Core Concept (Pattern / Topic)

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Repeatedly:

1. Remove first character from both strings
2. Append to result

Problem:

String deletion at beginning costs:

$O(n)$

Overall complexity becomes inefficient.

Optimized Thinking

Use two pointers:

$i \rightarrow \text{word1}$

$j \rightarrow \text{word2}$

Algorithm:

1. Traverse while either string still has characters
2. Add character from word1 if available



3. Add character from word2 if available
4. Continue until both finish

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

word1="abc"

word2="pqr"

Process:

a b c

↑

p q r

↑

Take:

a+p

ap+b+q

apbq+c+r

Output:

apbqcr

8. Pseudocode (Language Independent)

i=0

j=0

result=""

while i<len(word1)

or j<len(word2):

if i<len(word1):

add word1[i]



```
i++
```

```
if j<len(word2):
```

```
    add word2[j]
```

```
    j++
```

```
return result
```

9. Code Implementation

Python

```
class Solution:
```

```
    def mergeAlternately(
```

```
        self,
```

```
        word1,
```

```
        word2):
```

```
        i=0
```

```
        j=0
```

```
        result=[]
```

```
        while(
```

```
            i<len(word1)
```

```
            or
```

```
            j<len(word2)
```

```
        ):
```

```
            if i<len(word1):
```

```
                result.append(
```

```
                    word1[i]
```

```
                )
```



```
i+=1
```

```
if j<len(word2):
```

```
    result.append(  
        word2[j]  
    )
```

```
    j+=1
```

```
return "".join(result)
```

Java

```
class Solution {
```

```
    public String mergeAlternately(  
        String word1,  
        String word2) {
```

```
        int i=0;
```

```
        int j=0;
```

```
        StringBuilder result=  
            new StringBuilder();
```

```
        while(  
            i<word1.length()  
            ||  
            j<word2.length()  
        ){
```

```
            if(i<word1.length()){
```

```
                result.append(  
                    word1.charAt(i)  
                );
```



```
        i++;  
    }  
  
    if(j<word2.length()){  
  
        result.append(  
            word2.charAt(j)  
        );  
  
        j++;  
    }  
}  
  
return result.toString();  
}  
}
```

10. Time & Space Complexity

Time Complexity:

$O(n+m)$

Reason:

Each character is visited exactly once.

Space Complexity:

$O(n+m)$

Reason:

Result string stores all characters.

11. Common Mistakes / Edge Cases

1. Forgetting remaining characters

Wrong:

word1="ab"

word2="pqrs"

Output:

apbq



Correct:

apbqrs

2. Using string concatenation repeatedly

Wrong:

result += char

Can cause:

$O(n^2)$

behavior.

3. Missing unequal length cases

4. Missing single character case

word1="a"

word2="b"

12. Detailed Dry Run (Step-by-Step Table)

Input:

word1="ab"

word2="pqrs"

i	j	Character Added	Result
0	0	a,p	ap
1	1	b,q	apbq
2	2	r	apbqr
2	3	s	apbqrs

Final:

apbqrs

13. Common Use Cases (Real-Life / Interview)

Data stream merging

Combining message sequences

Interleaving data packets

Merging event timelines



14. Common Traps (Important!)

1. Forgetting leftover characters
2. Using expensive string concatenation
3. Incorrect loop conditions
4. Pointer out-of-bounds issues

15. Builds To (Related LeetCode Problems)

Progression:

- LC 1768 → Merge Strings Alternately
- LC 88 → Merge Sorted Array
- LC 21 → Merge Two Sorted Lists
- LC 97 → Interleaving String
- LC 392 → Is Subsequence

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
String concatenation	$O(n^2)$	$O(n)$	Repeated copies
Queue approach	$O(n+m)$	$O(n+m)$	Extra structure
Two Pointers + Builder	$O(n+m)$	$O(n+m)$	Optimal

17. Why This Solution Works (Short Intuition)

Two pointers ensure characters are picked in alternating order while preserving original sequence. When one string finishes, the remaining characters from the other string are appended automatically.

18. Variations / Follow-Up Questions

1. Merge three strings alternately
2. Merge linked lists alternately
3. Reverse alternate characters
4. Merge based on custom pattern
5. Merge multiple streams simultaneously
- 6.